

QB: Chapter 2.1 — Linked Lists

Unit 2 | 5×2 -mark | 5×5 -mark | 3×10 -mark | **Total: 13 questions**

Lectures: L16–L21 | BT Levels: BT1, BT2, BT3, BT4 | COs: CO1, CO2, CO5

Section A: 2-Mark Questions

Q1. [BT1 | CO2] Define a *singly linked list*. Name the two fields present in each node. What does the last node's link field contain, and why?

Q2. [BT1 | CO2] State two structural differences between a singly linked list and a doubly linked list. How does bidirectional traversal change the time complexity of deletion given a pointer to the node?

Q3. [BT2 | CO2] Describe a *header linked list*. What is the purpose of the header (dummy) node? Distinguish between a grounded header list and a circular header list.

Q4. [BT2 | CO2] Explain a *circular linked list*. How does its last node differ from a singly linked list? State one application that benefits from a circular structure.

Q5. [BT3 | CO2] Given the linked list: $\text{HEAD} \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow \text{NULL}$, trace the insertion of a new node containing 25 between nodes 20 and 30. Show the exact pointer updates required (two pointer assignments). Draw the list structure before and after insertion.

Section B: 5-Mark Questions

Q6. [BT3 | CO2] Implement `insert_at_beginning(data)` and `insert_at_end(data)` for a singly linked list. Trace both operations by inserting value 5 into the list $\text{HEAD} \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \text{NULL}$. Show the pointer assignments step by step for each insertion and draw the final list state.

Q7. [BT3 | CO2, CO5] Implement `linear_search(target)` for a singly linked list. Trace the search for value 30 in the list $\text{HEAD} \rightarrow 10 \rightarrow 30 \rightarrow 50 \rightarrow \text{NULL}$. Show which node is visited at each step. State the best-case and worst-case time complexity and give the input conditions that cause each.

Q8. [BT3 | CO2] Implement `delete_by_value(key)` for a singly linked list. Your implementation must handle three cases: deletion of the head node, deletion of a middle node, and deletion of the tail node. Trace deletion of value 20 from $\text{HEAD} \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$. Show the pointer update clearly and draw the resulting list.

Q9. [BT3 | CO2] Implement a doubly linked list with `insert_at_front(data)` and `delete_by_value(key)` operations. Trace inserting nodes 5, 10, 15 (in this order) using `insert_at_front`. Then trace `delete_by_value(10)`. For each operation, show all `prev` and `next` pointer assignments. Draw the list after all operations.

Q10. [BT4 | CO1, CO2] Construct a comparison table for arrays, singly linked lists, and doubly linked lists across the following six operations: random access by index, insertion at front, insertion at position *i* (middle), deletion at front, deletion at position *i* (middle), and extra memory used per element. Use O -notation for each cell. From the table, identify which structure is best suited for a playlist application where song tracks are frequently added and removed at the front, and random access by track number is not required.

Section C: 10-Mark Questions

Q11. [BT4 | CO1, CO2] (*Experiment E5 — Swap Nodes in Pairs, LeetCode #24*)

The "Swap Nodes in Pairs" problem requires swapping every two adjacent nodes in a linked list by relinking pointers — not by modifying node values.

(a) Design an iterative algorithm using a dummy header node. Describe the three pointer variables needed (`prev`, `curr`, `next_pair`) and explain the exact sequence of pointer reassignments required to swap one pair of nodes.

(b) Trace your algorithm on list `HEAD → 1 → 2 → 3 → 4 → NULL`. At each swap step, show the values of `prev`, `curr`, and `next_pair`, the pointer assignments performed, and the resulting list state. Show the state after all pairs are swapped.

(c) Analyse the time complexity ($O(n)$) and space complexity ($O(1)$) of the iterative approach. Compare with a recursive approach: write the recursive function and explain the trade-off in terms of call-stack depth and stack memory usage for a list of *n* nodes.

Q12. [BT4 | CO1, CO2]

A music player implements a playlist as a **circular doubly linked list**, where each node stores a song title and duration. The player supports four operations: `play_next()`, `play_prev()`, `add_song(title, duration)`, and `remove_song(title)`.

(a) Design the node structure and the `CircularDoublyLinkedList` class. Implement `play_next()` and `play_prev()` using a `current` pointer. Explain what happens when `current` reaches the tail during `play_next()`.

(b) Implement `add_song(title, duration)` to insert at the rear and `remove_song(title)` to delete a node by title. For each operation, draw the state of the list (showing `prev` and `next` pointers) before and after the operation.

(c) Derive the time complexity of all four operations. Justify why a circular doubly linked list is preferable to a singly linked list for this application — specifically addressing: backward navigation, deletion without traversing from head, and looping at list boundaries.

Q13. [BT4 | CO2]

A sparse polynomial $P(x) = 5x^4 + 3x^2 + 7$ is to be represented as a linked list storing only non-zero terms.

(a) Design the node structure with fields for coefficient, exponent, and next pointer. Represent $P(x)$ and $Q(x) = x^3 + 4x^2 + 5$ as linked lists. Draw each list showing all nodes in decreasing order of exponent.

(b) Implement polynomial addition for two such linked lists. Trace the addition of $P(x)$ and $Q(x)$ step by step: show which nodes are compared, whether exponents match or differ, and the state of the result list after each step.

(c) Compare linked list representation against array representation (index = exponent, value = coefficient) for polynomial storage. For a polynomial of degree 1000 with only 4 non-zero terms, compute the memory used by each representation (int = 4 bytes) and conclude which is more space-efficient. State the time complexity of polynomial addition under each representation.